

ACADEMIC TASK-1

CSE-316

(Operating System)

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Name – Shahnawaz Khan

Submitted to: Masroof Bashir



Realtime Process Scheduler Visualizer

Project Overview

The Realtime Process Scheduler Visualizer is an interactive tool that helps users understand how different CPU scheduling algorithms work. CPU scheduling plays a vital role in multitasking systems, determining which process gets access to the CPU at a given time. This project provides a real-time graphical representation of process execution, making it easier for students and professionals to visualize scheduling decisions. By allowing users to add, modify, and track processes dynamically, this tool serves as an educational and analytical resource. The project aims to enhance learning by enabling real-time visualization of scheduling behavior, demonstrating how different algorithms handle process execution, and analyzing their efficiency.

Module-Wise Breakdown

Module 1: User Interface Module

- Provides an intuitive GUI for user interaction.□
- Displays real-time process execution timelines.□
- Allows users to add, modify, or remove processes□
- CPU Utilization: Measures how effectively the CPU is utilized.□
- Throughput: Number of processes completed per unit of time□

Module 2: Scheduling Algorithm Module: Implements multiple scheduling algorithms such as:

- First-Come, First-Served (FCFS) – Processes are executed in the order they arrive.□
- Shortest Job Next (SJN) – The process with the shortest burst time executes first.□
- Round Robin (RR) – Each process gets a fixed time slice, ensuring fairness. Priority Scheduling – Processes with higher priority execute before others□

Module 3: Real-Time Data Processing Module

- Manages process execution updates dynamically.□
- Ensures correct order execution based on the selected algorithm.□
- Handles real-time calculations of scheduling metrics□

Module 4: Performance Metrics Module: Displays key performance indicators such as:

- Turnaround Time: The total time taken from process arrival to completion.□
- Waiting Time: The time a process spends waiting in the ready queue.□

- Users can select and switch between Repository Name: Realtime-Process-Scheduler-Visualizer different scheduling algorithms.
- Processes can be dynamically added, GithubLink: removed, or modified.
- Real-time visualization of CPU execution [Process](#) order. Calculation and display of [Scheduler-Visualizer](#)- scheduling performance metrics.
- Adjustable parameters such as burst time, *Conclusion and Future Scope* arrival time, and priority levels.
- Simulation speed control to observe This project provides an interactive way to understand Real execution at different rates. time process visualizer concepts through real time *Technology Recommendations* visualization. It offers a practical allowing users to experiment with different algorithms and approach to learning by
- *Programming Language:*□ analyze their impact on process execution ○ JavaScript (for web-based visualization, if *References* applicable) *Libraries and tools* Documentation: [https://](https://developer.mozilla.org/en-US/docs/Web/JavaScript) ○ HTML (front end)
 - CSS (Design) [https://](https://developer.mozilla.org/en-US/docs/Web/JavaScript)

Other Tools:

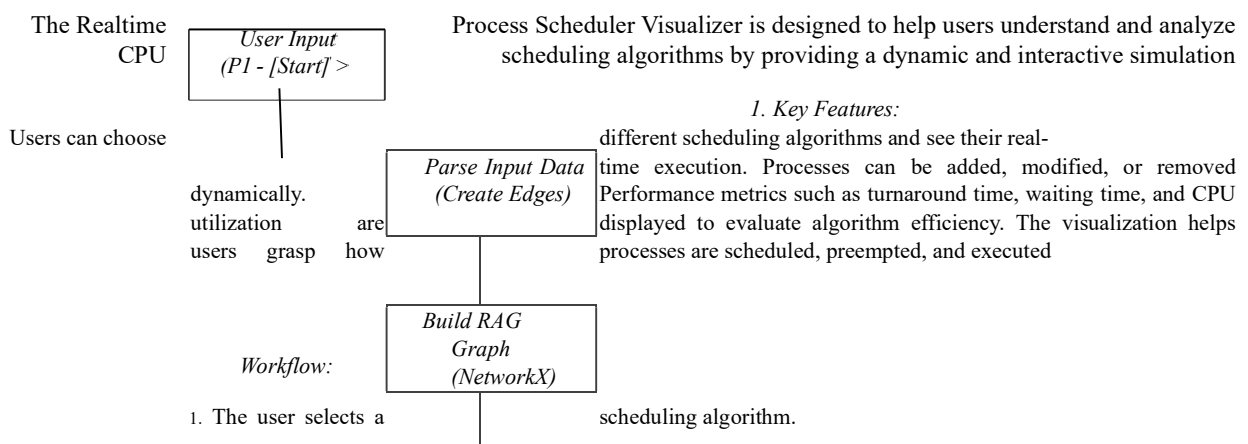
mukul2310.github.io/cpu-scheduler-visualiser/

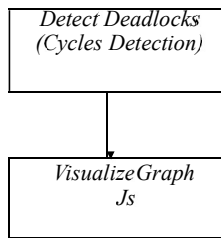
- GitHub: For version control, collaboration, and project tracking. 1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).
- Jupyter Notebook: For algorithm testing and Operating System Concepts. Wiley. debugging. 2. Stallings, W. (2017). Operating Systems: Internals and ○ Draw.io / Lucidchart: To create flow diagrams Design Principles. Pearson.
- and architecture designs 3. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems. Pearson.

Appendix

Flow Diagram Report

AIDetailed Breakdown of the Project-Generated Project Elaboration/Breakdown





2. Processes are added with burst time, arrival time, and priority.
3. The visualizer schedules and executes processes in real-time.

4. The system updates process execution dynamically and calculates performance metrics.
5. The output is displayed graphically with a Gantt chart. This tool is highly beneficial for students, researchers, and professionals who want to explore how different scheduling techniques work in practice

Problem Statement

Process scheduling is a core aspect of operating system design, ensuring efficient CPU utilization and system performance. Understanding how different scheduling algorithms work can be challenging, especially when relying on static diagrams and theoretical explanations.

- This project aims to provide a real-time, interactive scheduling simulation that allows users to:
- Visualize scheduling decisions dynamically.
- Compare different scheduling algorithms and analyze their efficiency.
- Interactively modify process parameters to see their impact on execution.
- Bridge the gap between theoretical learning and practical understanding

Solution/Code

```

1  <DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width">
6    <title>Process Visualizer</title>
7    <link rel="stylesheet" href="main.css">
8    <script src="https://cdn.jsdelivr.net/npm/chart.js">
9  </script>
10 </head>
11 <body>
12   <h2>Real-Time Process Visualizer</h2>
13   <div>
14     <div>
15       <div>
16         <div>
17           <div>
18             <div>
19               <div>
20                 <div>
21                   <div>
22                     <div>
23                       <div>
24                         <div>
25                           <div>
26                             <div>
27                               <div>
28                                 <div>
29                                   <div>
30                                     <div>
31                                       <div>
32                                         <div>

```

```

1  let processes = [];
2  let chartInstance = null;

3  function toggleInputs() {
4    let algorithm = document.getElementById("algorithm").value;
5    document.getElementById("priorityLabel").classList.toggle("hidden", algorithm !== "Priority");
6    document.getElementById("priority").classList.toggle("hidden", algorithm !== "Priority");
7    document.getElementById("quantumLabel").classList.toggle("hidden", algorithm !== "RoundRobin");
8    document.getElementById("quantum").classList.toggle("hidden", algorithm !== "RoundRobin");
9  }

10 function addProcess() {
11   let arrival = parseInt(document.getElementById("arrival").value);
12   let burst = parseInt(document.getElementById("burst").value);
13   let priority = document.getElementById("priority").value ? parseInt(document.getElementById("priority").value) : 0;

14   if (isNaN(arrival) || isNaN(burst) || burst <= 0) {
15     alert("Enter valid values!");
16     return;
17   }

18   let process = { id: "P" + (processes.length + 1), arrival, burst, priority };
19   processes.push(process);
20   displayProcesses();
21 }

22 function displayProcesses() {
23   let table = document.getElementById("processTable");
24   table.innerHTML = `
25     <tr>
26       <th>Process ID</th>
27       <th>Arrival Time</th>
28       <th>Burst Time</th>

```

```

1  /* Import Google Fonts */
2  @import url('https://fonts.googleapis.com/css2?family=Quicksand:wght@400;600&display=swap');

3  /* General Styles */
4  body {
5    font-family: 'Quicksand', sans-serif;
6    font-size: 1.2em;
7    color: #007acc;
8    background-color: #f0f0f0;
9    padding: 20px;
10 }

11 /* Headings */
12 h2 {
13   font-size: 2em;
14   font-weight: bold;
15   text-align: center;
16   margin-bottom: 20px;
17 }

18 h3 {
19   font-size: 1.5em;
20   font-weight: bold;
21   text-align: center;
22   margin-bottom: 20px;
23 }

24 /* Form Elements */
25 input, button {
26   border: 1px solid #ccc;
27   padding: 10px;
28   width: 100%;
29 }

30 button {
31   background-color: #007acc;
32   color: white;
33   border: none;
34 }

35 /* Table */
36 table {
37   width: 100%;
38   border-collapse: collapse;
39 }

40 table tr {
41   background-color: #f0f0f0;
42 }

43 table th {
44   padding: 10px;
45 }

46 table td {
47   padding: 10px;
48 }

```

Resource Allocation Graph Code:

```
# main.css | JS scripts X | main.html | README.md
JS scripts > ...
46 function runScheduler() {
47   //
48   if (algorithm === "FCFS") results = fcfs();
49   else if (algorithm === "SJF") results = sjf();
50   else if (algorithm === "RoundRobin") results = roundRobin();
51   else if (algorithm === "Priority") results = priorityScheduling();
52
53   processes = results;
54   displayResults();
55   visualizeGanttChart();
56   visualizeGraph();
57 }
58
59 function fcfs() {
60   let sorted = [...processes].sort((a, b) => a.arrival - b.arrival);
61   return calculateCompletion(sorted);
62 }
63
64 function sjf() {
65   let sorted = [...processes].sort((a, b) => (a.arrival - b.arrival) || (a.burst - b.burst));
66   return calculateCompletion(sorted);
67 }
68
69 function roundRobin() {
70   let quantum = parseInt(document.getElementById("quantum").value);
71   let queue = [...processes].sort((a, b) => a.arrival - b.arrival);
72   let time = 0, results = [];
73   let remainingBurst = queue.map(p => ({ ...p, remaining: p.burst }));
74
75   while (remainingBurst.length > 0) {
76     let p = remainingBurst.shift();
77     let executeTime = Math.min(p.remaining, quantum);
78     p.remaining -= executeTime;
79     results.push({ ...p, completion: time + executeTime });
80     time += executeTime;
81     if (p.remaining > 0) remainingBurst.push(p);
82   }
83   return results;
84 }
```

```
main.css | JS scripts X | main.html | README.md
JS scripts > ...
function displayResults() {
  //
}

function visualizeGanttChart() {
  let gantt = document.getElementById("gantt-chart");
  gantt.innerHTML = "";

  let time = 0;
  processes.forEach(p => {
    let box = document.createElement("div");
    box.className = "gantt-box";
    box.innerText = `${p.id} (${time} - ${p.completion})`;
    gantt.appendChild(box);
    time = p.completion;
  });
}

function visualizeGraph() {
  let ctx = document.getElementById("chart").getContext("2d");
  let labels = processes.map(p => p.id);
  let completionTimes = processes.map(p => p.completion);
  let turnaroundTimes = processes.map(p => p.turnaround);
  let waitingTimes = processes.map(p => p.waiting);

  if (chartInstance) chartInstance.destroy();
  chartInstance = new Chart(ctx, {
    type: "bar",
    data: {
      labels,
      datasets: [
        { label: "Completion Time", data: completionTimes, backgroundColor: "blue" },
      ],
    },
  });
}
```

